SQL and Database Fundamentals

Complete Notes with Examples and Diagrams

Database • DBMS • RDBMS • NoSQL • Hierarchy • Keys • Data Types • DDL • DML • DQL

Prepared for: Moses Putta

October 2026

Table of Contents

1. What is Data? 4

2. What is a Database? 4

3. What is a DBMS? 4

4. Server and Database Server 4

5. What is SQL? 5

6. Why SQL? 5

7. Types of Databases 6

8. Relational Database (RDBMS) 6

9. NoSQL Databases 7

9.1 Key-Value Database . . . 7

9.2 Document Database . . . 7

9.3 Column-Based (Wide-Column) Database . . . 8

9.4 Graph Database . . . 8

10. Database Hierarchy (Structure) 8

11. Primary Key 9

12. Cell / Value 10

13. Data Types 10

13.1 Numeric . . . 10

13.2 Text (String) . . . 11

13.3 Date and Time . . . 11

13.4 Others . . . 11

14. Constraints (Rules on Columns) 12

15. Categories of SQL Commands 12

16. DDL: Data Definition Language 13

16.1 CREATE: make new objects . . . 13

16.2 ALTER: modify an existing table . . . 13

16.3 DROP: delete the object permanently . . . 13

16.4 TRUNCATE: remove all rows, keep structure . . . 13

17. DML: Data Manipulation Language 14

17.1 INSERT: add new rows . . . 14

17.2 UPDATE: modify existing rows . . . 14

17.3 DELETE: remove rows . . . 14

18. DQL: Data Query Language (SELECT) **15**

19. DCL and TCL (Quick Overview) **15**

20. Complete End-to-End Example **16**

21. Quick Revision Cheat Sheet **16**

22. Practice Questions **17**

1. What is Data?

Data is raw facts: a name, a number, a date. Example: `Moses`, `25`, `Hyderabad`, `2026-10-01`.

On its own, data has no structure. When data is organized so you can store, find, and update it easily, you get a **database**.

2. What is a Database?

A **database** is an organized collection of data stored electronically so it can be easily accessed, managed, and updated.

Real-life analogy

Real world	Database world
Library building	Database server
A section in the library (Science, History)	Database
A bookshelf	Table
One book entry card	Row
Info on the card (title, author)	Columns

Where databases are used

- Amazon stores products, customers, and orders
- Banks store accounts and transactions
- Hospitals store patients and appointments
- WhatsApp stores messages and contacts

3. What is a DBMS?

DBMS = Database Management System. It is the **software** that lets you create, store, read, update, and delete data in a database. You never touch the data files directly. You talk to the DBMS, and it handles the files.

```
You / Application  ---->  DBMS (software)  ---->  Data files on disk
```

Function	Meaning
Storage	Saves data on disk efficiently
Retrieval	Finds data fast
Security	Controls who can see or change what (users, passwords, permissions)
Backup and recovery	Restores data after a crash
Concurrency	Many users work at the same time without corrupting data
Integrity	Enforces rules (e.g., no duplicate IDs)

Examples of DBMS software: MySQL, PostgreSQL, Oracle, SQL Server, MongoDB, Redis, Cassandra, Neo4j.

4. Server and Database Server

Server: a computer (physical or virtual) that provides a service to other computers, called clients.

Server type	Service it provides
Web server (Nginx, Apache)	Serves websites
Mail server	Handles email
File server	Stores shared files
Database server	Stores and serves data

Database Server: a server that has a DBMS installed and running on it. It listens on a network port and answers requests from applications.

```
[Client / App]  --- request (SQL query) --->  [Database Server]
                                              (DBMS running here)
[Client / App]  <--- result (data) ----------
```

Default ports to remember

DBMS	Default port
MySQL	3306
PostgreSQL	5432
SQL Server	1433
Oracle	1521
MongoDB	27017
Redis	6379

Typical 3-tier application

```
Browser   -->   Application Server (Java/Node)   -->   Database Server (MySQL)
(user)          (business logic)                      (stores the data)
```

Note: "Database server" can mean the machine or the DBMS software running on it. People use the term for both.

5. What is SQL?

SQL = Structured Query Language (pronounced "S-Q-L" or "sequel"). SQL is the **language** you use to talk to a relational database. It lets you tell the DBMS what to do.

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Meaning: "Show me the name and salary of employees who earn more than 50000."

- SQL is a language, not a database and not software.
- MySQL, PostgreSQL, and Oracle are DBMS software that understand SQL.
- SQL is a standard (ANSI/ISO), so most commands work across all of them, with small differences.

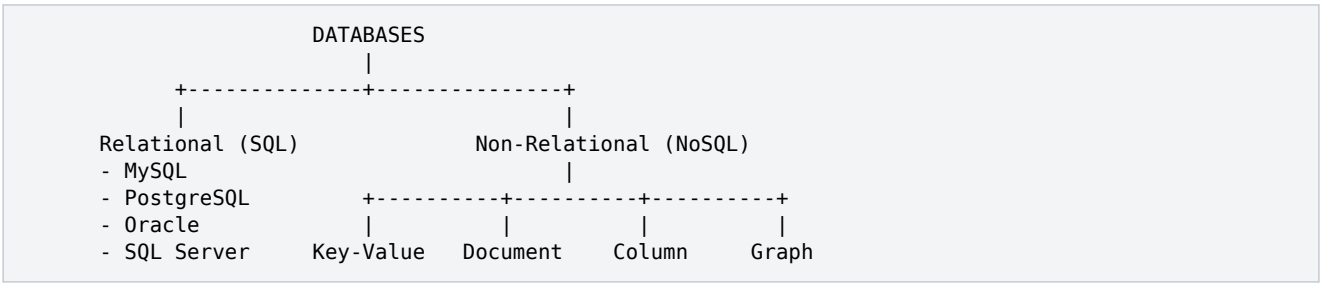
6. Why SQL?

Reason	Explanation
Easy to learn	Reads like English: SELECT, FROM, WHERE
Industry standard	Used by almost every company
Handles huge data	Millions or billions of rows
Powerful	Filter, sort, join, group, and calculate in one query
Fast	Databases optimize queries with indexes
Secure	Permission control with GRANT and REVOKE
Consistent	Rules (constraints) keep data correct
Works everywhere	Same skill for MySQL, PostgreSQL, Oracle, and others
Career value	Needed for DevOps, backend, data, QA, security, and cloud roles

DevOps and DevSecOps relevance

- Apps on Kubernetes connect to RDS (MySQL/PostgreSQL)
- You troubleshoot DB connection errors
- You handle backups and secrets for DB credentials
- You must understand SQL injection, a top security vulnerability

7. Types of Databases



8. Relational Database (RDBMS)

RDBMS = Relational Database Management System. Data is stored in **tables** (rows and columns). Tables are **related** to each other using keys.

Example: two tables related by dept_id

departments

dept_id (PK)	dept_name
10	DevOps
20	Security

employees

emp_id (PK)	name	salary	dept_id (FK)
1	Ravi	60000	10
2	Sita	70000	20
3	Kiran	55000	10

```
departments.dept_id  <---- relationship ---->  employees.dept_id
```

- **PK** = Primary Key (unique ID of a row)
- **FK** = Foreign Key (a column pointing to a PK in another table)

ACID properties

Letter	Meaning	Example
Atomicity	All or nothing	Money transfer: both debit and credit happen, or neither
Consistency	Data always follows rules	Balance can't become invalid
Isolation	Users don't interfere with each other	Two people booking the same seat
Durability	Saved data survives a crash	Power cut after commit, data is safe

Best for: banking, e-commerce orders, ERP, anything needing accuracy and relationships.
Examples: MySQL, PostgreSQL, Oracle, SQL Server, MariaDB, AWS RDS, Aurora.

9. NoSQL Databases

NoSQL = "Not Only SQL". These databases don't use the table-and-row model (or not only that). They are built for flexibility, huge scale, and speed.

Feature	SQL (Relational)	NoSQL
Structure	Fixed schema (tables)	Flexible schema
Data model	Rows and columns	Key-value, document, column, graph
Scaling	Mostly vertical (bigger server)	Mostly horizontal (more servers)
Relationships	Strong (joins)	Weak or none
Query language	SQL	Varies per database
Consistency	Strong (ACID)	Often eventual (BASE)
Best for	Structured, related data	Big data, real-time, unstructured data

9.1 Key-Value Database

Stores data as simple **key : value** pairs, like a dictionary.

```
KEY              VALUE
"user:101"    -->  "Moses"
"session:abc" -->  "{logged_in: true}"
"cart:55"     -->  "[shirt, shoes]"

SET user:101 "Moses"
GET user:101        ->  "Moses"
```

- Extremely fast (lookup by key); no complex queries
- **Use cases:** caching, session storage, leaderboards, rate limiting
- **Examples:** Redis, DynamoDB, Memcached

9.2 Document Database

Stores data as **documents** (JSON-like). Each document can have a different structure.

```
{ "_id": 1,
  "name": "Moses",
  "skills": ["Docker", "Kubernetes", "Jenkins"],
  "address": { "city": "Hyderabad", "state": "Telangana" } }

{ "_id": 2,
  "name": "Sita",
  "email": "sita@mail.com" }        <-- different fields, and that is allowed
```

- **Use cases:** product catalogs, user profiles, content management, APIs
- **Examples:** MongoDB, CouchDB, Firestore

9.3 Column-Based (Wide-Column) Database

Stores data **by columns** instead of by rows. Very efficient for reading a few columns across billions of rows (analytics).

```
Row storage (traditional):
  [1, Ravi, 60000]  [2, Sita, 70000]  [3, Kiran, 55000]

Column storage:
  id:      [1, 2, 3]
  name:    [Ravi, Sita, Kiran]
  salary:  [60000, 70000, 55000]
```

To compute the average salary, the database reads only the salary column and skips the rest.

- **Use cases:** analytics, time-series, IoT, huge write volumes, logging
- **Examples:** Cassandra, HBase, Google Bigtable (Redshift and ClickHouse use columnar storage)

9.4 Graph Database

Stores data as **nodes** (things) and **edges** (relationships between things).

```
   (Moses) ---FRIENDS_WITH---> (Ravi)
      |                          |
   WORKS_AT                   WORKS_AT
      |                          |
      v                          v
   (Asheer IT)  <----------  (Asheer IT)

MATCH (a:Person)-[:FRIENDS_WITH]->(b:Person) RETURN a, b     (Cypher, not SQL)
```

- Excellent when relationships are the main point
- **Use cases:** social networks, recommendation engines, fraud detection, network topology
- **Examples:** Neo4j, Amazon Neptune

Comparison of the 4 NoSQL types

Type	Data shape	Strength	Example DB	Use case
Key-Value	key : value	Speed	Redis	Cache, sessions
Document	JSON documents	Flexibility	MongoDB	Profiles, catalogs
Column	Column families	Big analytics and writes	Cassandra	IoT, logs
Graph	Nodes and edges	Relationships	Neo4j	Social, fraud

10. Database Hierarchy (Structure)

```
SERVER
  |__ DATABASE
        |__ SCHEMA
              |__ TABLE
                    |__ COLUMNS  (vertical; define the type of data)
                    |__ ROWS     (horizontal; one record each)
                          |__ CELL (row x column intersection = value)
```

Level	Meaning	Analogy	Example
Server	Machine/software hosting databases	Whole company building	mysql-server-01
Database	Container for related data	One department	company_db
Schema	Logical group inside a database	A folder or section	hr, sales
Table	Stores one type of entity	One register or sheet	employees
Column	One attribute (field)	A column heading	name, salary
Row	One complete record (tuple)	One person's entry	Row of Ravi
Cell	One single value	One box in Excel	60000

Note: In PostgreSQL / SQL Server / Oracle, a schema is a real layer inside a database (default is `public` in PostgreSQL and `dbo` in SQL Server). In MySQL, schema and database mean the same thing. "Schema" is also used for the design/blueprint of a database.

Visual example of one table: employees

```
           COLUMNS (attributes)
         +--------+--------+---------+------------+
         | emp_id |  name  | salary  | join_date  |   <- column names (header)
         +--------+--------+---------+------------+
ROW 1 > |   1    | Ravi   | 60000   | 2024-01-15 |
ROW 2 > |   2    | Sita   | 70000   | 2023-06-10 |
ROW 3 > |   3    | Kiran  | 55000   | 2025-03-01 |
         +--------+--------+---------+------------+
             ^
       PRIMARY KEY column
```

- **Cell:** the value at row 2, column `name` is `Sita`.
- **Column:** `salary` is the whole vertical set (60000, 70000, 55000).
- **Row (record/tuple):** row 1 is (1, Ravi, 60000, 2024-01-15).

```
CREATE DATABASE company_db;          -- create a database
USE company_db;                      -- select it (MySQL)

CREATE TABLE employees (             -- create a table
    emp_id    INT PRIMARY KEY,
    name      VARCHAR(50),
    salary    DECIMAL(10,2),
    join_date DATE
);
```

11. Primary Key

A **primary key (PK)** is a column (or set of columns) that **uniquely identifies each row**.

Rule	Meaning
Unique	No two rows can have the same value
NOT NULL	Cannot be empty
One per table	Only one PK (can be multiple columns, called a composite key)
Stable	Should rarely change

emp_id (PK)	name
1	Ravi
2	Sita
1	Kiran <-- ERROR, duplicate PK

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,   -- auto generates 1,2,3...
    name   VARCHAR(50)
);
```

Related keys

Key	Purpose	Example
Primary Key	Unique row ID	emp_id
Foreign Key	Links to another table's PK	employees.dept_id -> departments.dept_id
Unique Key	Unique values, but NULL allowed	email
Composite Key	PK made of 2+ columns	(order_id, product_id)
Candidate Key	Any column that could be a PK	emp_id, email

12. Cell / Value

A **cell** is the intersection of one row and one column. It holds **one single value**.

```
              name      salary
 row 1  -->  [Ravi]    [60000]     <- each [ ] is a cell
 row 2  -->  [Sita]    [70000]
```

- A cell holds one atomic value (not a list) in a properly designed relational table.
- A cell can be **NULL**, meaning "no value / unknown". NULL is not zero and not an empty string.

```
-- Update one cell: row where emp_id = 2, column salary
UPDATE employees SET salary = 75000 WHERE emp_id = 2;
```

13. Data Types

Every column must have a **data type**. It tells the database what kind of data is allowed and how much space to use.

13.1 Numeric

Type	Description	Example
TINYINT	Very small whole number (-128 to 127)	age, flag

Type	Description	Example
SMALLINT	Small integer	quantity
INT / INTEGER	Whole number (about +/- 2.1 billion)	emp_id, 1001
BIGINT	Very big whole number	phone numbers, big IDs
DECIMAL(p,s) / NUMERIC	Exact decimal. p = total digits, s = digits after the point	DECIMAL(10,2) = 12345678.90
FLOAT, DOUBLE	Approximate decimal (fast, small rounding errors)	scientific data

Important: Use DECIMAL for money. Never use FLOAT for money, because rounding errors occur.

13.2 Text (String)

Type	Description	Example
CHAR(n)	**Fixed** length. Pads with spaces.	CHAR(2) = 'IN', 'US'
VARCHAR(n)	**Variable** length, up to n	VARCHAR(50) = 'Moses'
TEXT	Long text	descriptions, blogs

```
CHAR(10)     'Ravi'  -->  stored as 'Ravi      '   (10 bytes always)
VARCHAR(10)  'Ravi'  -->  stored as 'Ravi'        (4 bytes + small overhead)
```

Use CHAR for fixed-size values (country code). Use VARCHAR for names, emails, etc.

13.3 Date and Time

Type	Format	Example
DATE	YYYY-MM-DD	2026-10-01
TIME	HH:MM:SS	14:30:00
DATETIME	Date and time	2026-10-01 14:30:00
TIMESTAMP	Date and time (often auto-updates, timezone-aware)	2026-10-01 14:30:00
YEAR	4-digit year (MySQL)	2026

13.4 Others

Type	Description	Example
BOOLEAN / BOOL	TRUE or FALSE	is_active
ENUM	One value from a fixed list (MySQL)	ENUM('Male','Female')
BLOB	Binary data (images, files)	profile picture
JSON	JSON documents	settings
UUID	Unique ID string (PostgreSQL)	a0eebc99-...

```
CREATE TABLE employees (
    emp_id      INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(50)    NOT NULL,
    gender      CHAR(1),
    salary      DECIMAL(10,2),
    is_active   BOOLEAN        DEFAULT TRUE,
    join_date   DATE,
    created_at  TIMESTAMP      DEFAULT CURRENT_TIMESTAMP
);
```

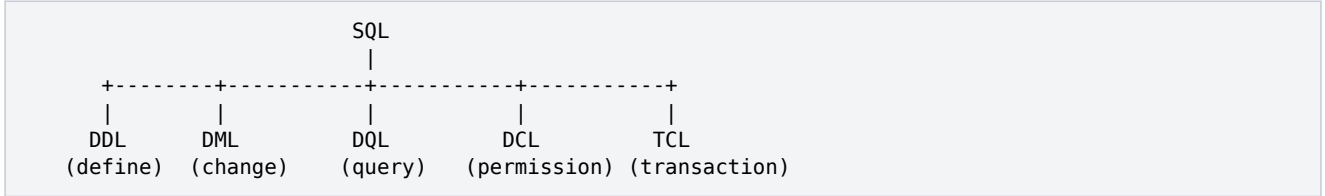
14. Constraints (Rules on Columns)

Constraint	Meaning	Example
PRIMARY KEY	Unique and not null	emp_id INT PRIMARY KEY
FOREIGN KEY	Must match a PK in another table	FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
NOT NULL	Cannot be empty	name VARCHAR(50) NOT NULL
UNIQUE	No duplicates	email VARCHAR(100) UNIQUE
DEFAULT	Value used if none given	status VARCHAR(10) DEFAULT 'active'
CHECK	Condition must be true	CHECK (salary > 0)

```
CREATE TABLE departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE employees (
    emp_id   INT PRIMARY KEY,
    name     VARCHAR(50) NOT NULL,
    email    VARCHAR(100) UNIQUE,
    salary   DECIMAL(10,2) CHECK (salary > 0),
    dept_id  INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

15. Categories of SQL Commands



Category	Full name	Purpose	Commands
DDL	Data Definition Language	Define or change **structure**	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Change **data** inside tables	INSERT, UPDATE, DELETE
DQL	Data Query Language	**Read** data	SELECT
DCL	Data Control Language	Permissions	GRANT, REVOKE
TCL	Transaction Control Language	Control transactions	COMMIT, ROLLBACK, SAVEPOINT

Some books place SELECT under DML. Both are accepted. Listing it separately as DQL is also common.

16. DDL: Data Definition Language

DDL works on the **structure** (databases, tables, columns), not the data rows.

16.1 CREATE: make new objects

```
CREATE DATABASE company_db;

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50) NOT NULL,
    age        INT,
    city       VARCHAR(30)
);
```

16.2 ALTER: modify an existing table

Task	Syntax
Add a column	`ALTER TABLE students ADD email VARCHAR(100);`
Delete a column	`ALTER TABLE students DROP COLUMN city;`
Change data type	`ALTER TABLE students MODIFY age SMALLINT;` (MySQL)
Rename a column	`ALTER TABLE students RENAME COLUMN name TO full_name;`
Rename the table	`ALTER TABLE students RENAME TO learners;`
Add a constraint	`ALTER TABLE students ADD CONSTRAINT uq_email UNIQUE (email);`

PostgreSQL uses `ALTER COLUMN age TYPE SMALLINT` instead of `MODIFY`.

```
 BEFORE ADD email :  student_id | name | age | city
 AFTER  ADD email :  student_id | name | age | city | email
```

Existing rows get NULL in the new column.

16.3 DROP: delete the object permanently

```
DROP TABLE students;          -- deletes table AND its data AND structure
DROP DATABASE company_db;     -- deletes the entire database
```

> **Warning:** DROP cannot be undone. Be very careful in production.

16.4 TRUNCATE: remove all rows, keep structure

```
TRUNCATE TABLE students;
```

DELETE vs TRUNCATE vs DROP

	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL

	DELETE	TRUNCATE	DROP
Removes	Selected or all rows	All rows	Table itself (structure and data)
WHERE allowed	Yes	No	No
Structure remains	Yes	Yes	No
Speed	Slower	Fast	Fast
Rollback	Yes (in a transaction)	Usually no	No

17. DML: Data Manipulation Language

DML works on the **data (rows)** inside tables. Sample table used in the examples:

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50),
    age        INT,
    city       VARCHAR(30)
);
```

17.1 INSERT: add new rows

```
-- One row
INSERT INTO students (student_id, name, age, city)
VALUES (1, 'Ravi', 22, 'Hyderabad');

-- Multiple rows
INSERT INTO students (student_id, name, age, city) VALUES
(2, 'Sita',  21, 'Vijayawada'),
(3, 'Kiran', 23, 'Guntur'),
(4, 'Anil',  22, 'Hyderabad');
```

Table now:

student_id	name	age	city
1	Ravi	22	Hyderabad
2	Sita	21	Vijayawada
3	Kiran	23	Guntur
4	Anil	22	Hyderabad

17.2 UPDATE: modify existing rows

```
UPDATE students
SET age = 24, city = 'Chennai'
WHERE student_id = 3;
```

student_id	name	age	city
3	Kiran	**24**	**Chennai**

> **Warning:** Always use WHERE with UPDATE. Without it, every row is updated.

17.3 DELETE: remove rows

```
DELETE FROM students WHERE student_id = 4;   -- deletes only Anil
DELETE FROM students;                        -- deletes ALL rows (no WHERE)
```

18. DQL: Data Query Language (SELECT)

`SELECT` **reads** data without changing it. It is the most-used command.

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column
LIMIT n;
```

```
SELECT * FROM students;                                  -- all columns, all rows
SELECT name, city FROM students;                         -- specific columns
SELECT * FROM students WHERE city = 'Hyderabad';         -- filter rows
SELECT * FROM students WHERE age >= 22 AND city = 'Hyderabad';
SELECT * FROM students ORDER BY age DESC;                -- sort high to low
SELECT * FROM students LIMIT 2;                          -- first 2 rows
SELECT * FROM students WHERE name LIKE 'R%';             -- names starting with R
SELECT DISTINCT city FROM students;                      -- unique values

SELECT COUNT(*) FROM students;                           -- number of rows
SELECT AVG(age) FROM students;                           -- average age
SELECT MAX(age), MIN(age) FROM students;

SELECT city, COUNT(*) AS total
FROM students
GROUP BY city;
```

Result of the GROUP BY query:

city	total
Hyderabad	2
Vijayawada	1
Guntur	1

Order of writing vs. order of execution

```
 Written :  SELECT > FROM > WHERE > GROUP BY > HAVING > ORDER BY > LIMIT
 Executed:  FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT
```

JOIN (combining related tables)

```
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

name	dept_name
Ravi	DevOps
Sita	Security
Kiran	DevOps

19. DCL and TCL (Quick Overview)

DCL: permissions

```
GRANT  SELECT, INSERT ON company_db.employees TO   'app_user'@'%';
REVOKE INSERT         ON company_db.employees FROM 'app_user'@'%';
```

TCL: transactions

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE acc_id = 2;
COMMIT;        -- save permanently
-- or ROLLBACK; -- undo everything since START TRANSACTION
```

20. Complete End-to-End Example

```
-- 1. DDL: create the structure
CREATE DATABASE school_db;
USE school_db;

CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name       VARCHAR(50) NOT NULL,
    age        INT,
    city       VARCHAR(30)
);

-- 2. DDL: modify the structure
ALTER TABLE students ADD email VARCHAR(100);

-- 3. DML: add data
INSERT INTO students (name, age, city, email)
VALUES ('Ravi', 22, 'Hyderabad', 'ravi@mail.com'),
       ('Sita', 21, 'Vijayawada', 'sita@mail.com');

-- 4. DQL: read data
SELECT * FROM students;

-- 5. DML: change data
UPDATE students SET city = 'Chennai' WHERE name = 'Sita';

-- 6. DML: delete data
DELETE FROM students WHERE name = 'Ravi';

-- 7. DDL: remove the table
DROP TABLE students;
```

21. Quick Revision Cheat Sheet

Concept	One-line answer
Database	Organized collection of data
DBMS	Software that manages databases
Database server	Server running a DBMS to serve data
SQL	Language to talk to relational databases
RDBMS	DBMS storing data in related tables
Key-value	key : value pairs (Redis)
Document	JSON-like documents (MongoDB)

Concept	One-line answer
Column-based	Data stored by columns (Cassandra)
Graph	Nodes and edges (Neo4j)
NoSQL	Non-relational, flexible, scalable
Hierarchy	Server > Database > Schema > Table > Columns/Rows > Cell
Primary key	Unique, not-null row identifier
Foreign key	Links one table to another
Cell	One value (row x column)
DDL	CREATE, ALTER, DROP, TRUNCATE
DML	INSERT, UPDATE, DELETE
DQL	SELECT
DCL	GRANT, REVOKE
TCL	COMMIT, ROLLBACK

22. Practice Questions

1. What is the difference between a database and a DBMS?

2. Why is DECIMAL better than FLOAT for storing money?

3. What is the difference between CHAR(10) and VARCHAR(10)?

4. Why can a table have only one primary key?

5. What is the difference between DELETE, TRUNCATE, and DROP?

6. Which NoSQL type is best for caching sessions? For social network relationships?

7. Write a query to show all employees with salary above 50000, highest first.

Answer to Q7:

```
SELECT * FROM employees WHERE salary > 50000 ORDER BY salary DESC;
```

Next topics to study: JOINs, GROUP BY / HAVING, indexes, normalization, subqueries.